

IPL Win-Probability & Score Prediction

What this system computes, why, and how each design choice maps onto — or departs from — the published literature. Written as standalone context for research-paper writing; assumes no prior knowledge of how the codebase came to exist.

1,146

matches (2008–2025)

273,503

deliveries

74

locked 2026 holdout matches

0.878

dynamic 2nd-innings AUC

−0.02

pre-match Brier Skill Score

CONTENTS

1. What this system does, in one paragraph	§1
2. Where this sits in the literature	§2
3. Data layer — src/data.py	§3
4. Team-strength layer — src/elo.py	§4
5. Match-level features — src/features.py	§5
6. The 24-dimensional per-ball game state	§6
7. Player-level features — src/player_features.py	§7
8. The core prediction pipeline — src/pipeline.py	§8
9. Temperature scaling — src/temperature_scaling.py	§9
10. Secondary model: causal Transformer + MC-Dropout	§10
11. Explainability — src/explainability.py	§11
12. Tournament simulation — src/tournament.py	§12
13. Evaluation-metric definitions — src/metrics.py	§13
14. Presentation layer — dashboard	§14
15. Test suite	§15
16. Results — the full numbers, consolidated	§16
17. Honest limitations	§17
18. Suggested framing for a research paper	§18

1 What this system does, in one paragraph

Given IPL ball-by-ball data (2008–2025) plus a locked-away 2026 holdout set, the system predicts two things — **who wins a match** and **what score a team will finish on** — at two different moments: **before a ball is bowled** (pre-match) and **as the match unfolds, ball by ball** (dynamic/in-game). Every prediction is a *calibrated probability*, not just a label, and every evaluation reports calibration (Brier score, Expected Calibration Error) alongside accuracy/AUC — a distinction the literature (Paper 31) shows matters enormously in practice.

⚠ A model picked by accuracy alone lost money at –35% ROI in a betting simulation; the calibration-selected model made +34.69%. The codebase treats calibration as a first-class design constraint, not an afterthought.

2 Where this sits in the literature

The 51 reference papers split into two philosophies:

	PAPERS 1–35 (STANDARD ML)	PAPERS 36–51 (BAYESIAN/PROBABILISTIC)
Output	Win/loss label	Probability distribution
Team strength	Engineered average (a feature)	Latent parameter, often with a prior
Temporal dynamics	Static feature table, one row per match	Dynamic — updates as new information arrives
Evaluation	Accuracy	Calibration + accuracy

This codebase sits between the two camps, closer to the probabilistic side than any of Papers 1–26, but without going fully Bayesian (no MCMC, no posterior distributions, no PyMC/Stan). Concretely, it **does**:

- adopt the probabilistic framing $P(\text{win} \mid \text{state at ball } t)$ for every t , not a single pre-match label (Paper 49 / Paper 8);
- treat calibration as a primary evaluation axis (BSS, ECE, reliability diagrams) throughout `src/metrics.py` and `src/pipeline.py` (Paper 31);
- use a team-strength layer estimated from historical results (Elo) rather than static per-match averages, in the spirit of Papers 36–44's "team ability" parameters, but via the chess Elo update rule;
- use SHAP for interpretability (Paper 29's contribution to the NBA literature), ported here to IPL pre-match features.

It **does not** implement a full Bayesian hierarchical model (no priors on team strength, no posterior sampling, no GP smoothing over the match as in Paper 49) and does not touch player-price/auction prediction or fantasy team selection.

This gap is itself a legitimate research angle: the codebase proves out the *evaluation methodology* the Bayesian papers argue for (calibration-first, walk-forward, no lookahead) on a genuinely large IPL dataset, but stops short of the full hierarchical/Bayesian treatment those same papers describe.

3 Data layer — `src/data.py`

Input is a single Excel workbook (`ipl_data.xlsx`), sheet `"Ball by Ball"` , one row per legal/illegal delivery across 1,146 matches (2008–2025), plus a separate locked 2026 holdout (`data/external_2026/*.csv` , 74 matches) that is **never touched** until the final evaluation step — the walk-forward discipline the Bayesian papers insist on.

Two responsibilities

1. Team-name normalisation (`NAME_MAP`) — IPL franchises have renamed or folded over 18 seasons (Delhi Daredevils → Delhi Capitals, Deccan Chargers → Sunrisers Hyderabad, etc.). Applied once, upstream of everything else, in `load_ball_by_ball` .

2. Match-table construction (`build_match_table`) — collapses delivery-level rows into one row per match: `team1` (batting first), `team2` (bowling first), `winner` , `toss_winner` / `toss_decision` , and the two binary toss-advantage features `toss_bat_first` / `toss_field_first` . This becomes the backbone `match_df` every other feature-engineering step joins against.

4 Team-strength layer — `src/elo.py`

Standard chess Elo ($K=32$, initial rating 1500), applied match-by-match in chronological order — the codebase's closest analogue to the Bayesian papers' "team ability" parameter, except estimated via the Elo point-update rule rather than a hierarchical posterior.

```
exp1 = 1 / (1 + 10^((elo2 - elo1) / 400))    # expected win prob for team1
elo1_new = elo1 + K * (actual_outcome - exp1) # actual_outcome ∈ {0, 1}
elo2_new = elo2 + K * ((1-actual_outcome) - (1-exp1))
```

Two entry points, sharing one `_update()` helper:

- `compute_elo(match_df)` — each match's **pre-match** rating for both teams (`elo1`, `elo2`, `elo_diff`) plus the final end-of-history rating dict, used as the walk-forward starting point for 2026 evaluation.
- `compute_elo_history(match_df)` — each team's full chronological rating trajectory, for the dashboard's Elo-over-time visualisation.



No shrinkage/prior. Every team starts at exactly 1500 regardless of how much historical data exists elsewhere in the league. A new franchise's first Elo value is a fixed constant, not a partially-pooled estimate — a concrete extension point: replace the point-estimate Elo update with a Bayesian rating system (e.g. TrueSkill) and measure whether shrinkage improves early-season predictions for young franchises (GT, LSG).

5 Match-level features — `src/features.py`

Five scalar helper functions plus two walk-forward accumulators. Every function in this module explicitly enforces **no-lookahead**: at the point a feature is computed for match i , it has only seen the outcomes of matches $0..i-1$.

- `balls_remaining`, `runs_needed`, `crr` (current run rate), `rrr` (required run rate), `phase` (Powerplay 0–6 / Middle 7–15 / Death 16–19, encoded 0/1/2) — the standard in-game state primitives every dynamic model is built from.
- `compute_form_h2h(match_df, window=5)` — walk-forward recent form (rolling win-rate over each team's last 5 matches, defaulting to 0.5 with no history) and raw head-to-head win rate (also defaulting to 0.5 with no shared history).
- `compute_h2h_beta(match_df, alpha=2, beta=2)` — **the one place in the codebase that actually does what the Bayesian papers describe**: a Beta(2,2) conjugate prior over head-to-head win rate, posterior mean $(\text{wins} + 2) / (n + 4)$. With zero prior meetings this is exactly 0.5 (uninformative); with more meetings it shrinks smoothly toward the raw empirical rate.

6 The 24-dimensional per-ball game state

The densest feature-engineering module, and the one with a real, documented, test-caught bug fix baked into its design.

 **The bug (now fixed, permanently regression-tested):** early implementations paired a "before this ball" numerator (e.g. `score_before`) with an "including this ball" denominator (`legal_balls_total`), which silently **halved the reported run rate on ball 2 of every innings** (and similarly distorted `boundary_rate` , `dot_rate`). The fix introduces `legal_balls_before` , paired consistently everywhere a ratio needs a pre-ball denominator.

The 24 features (indices)

```
0  over / 20
1  legal_balls_total / 120
2  innings==2 flag
3-5 phase one-hot
6  score_before / 250
7  wickets_before / 10
8  run_rate (clipped 15/ov)
9  runs in last completed over/30
10 wickets in last 5 overs / 5
11 partnership_runs / 150
12 partnership_balls / 60
13 batter_sr_innings / 200
14 batter_balls_innings / 120
15 bowler_econ_innings (clipped 20/ov)
16 bowler_wkts_innings / 10
17 boundary_rate so far
18 dot_rate so far
19 runs_required / 200 (inn.2 only)
20 balls_remaining / 120 (inn.2 only)
21 required_rr (clipped 36/ov, inn.2)
22 toss_won_bat
23 innings==2 flag (duplicate of #2)
```

Computed with vectorised pandas `groupby` / `cumsum` / `shift` operations rather than a row-by-row loop — a deliberate performance choice given 273,503 deliveries.

None of Papers 1–26 discuss this pre-ball/post-ball timing subtlety at all. This codebase's fix (and the regression test that catches it) is a concrete, reportable methodological contribution — a worked example of how silently-wrong feature timing degrades exactly the early-innings predictions that matter most for in-game win-probability graphics.

7 Player-level features — `src/player_features.py`

Two walk-forward accumulators, both **season-boundary** (not match-boundary) in granularity:

- `compute_rolling_player_stats` — per-player batting (runs, balls, SR, boundary rate, phase-specific SR) and bowling (economy, wickets, dot rate, phase-specific economy) stats.
- `compute_matchup_features` — same walk-forward pattern, keyed by `(batter, bowler)` pair: balls faced, runs scored, dismissals, boundary rate, dot rate, strike rate in that specific matchup.

💡 **This module is additive and currently unconsumed** — its outputs don't feed into any trained model in `src/pipeline.py`. This is the single clearest gap between what the codebase computes and what the literature describes as valuable signal — probably the highest-leverage extension for a research paper: the raw ingredients for a player-level, matchup-aware win-probability model already exist and are tested; they're simply not wired into a model yet.

8 The core prediction pipeline — `src/pipeline.py`

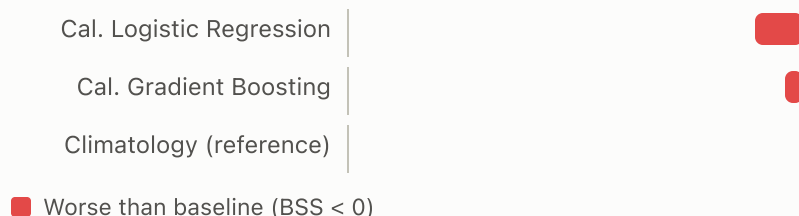
The largest module and the one that actually trains and evaluates models. Five distinct prediction tasks, each evaluated with a **temporal train/test split** (train ≤ 2020 , test > 2020) rather than a random split.

8.1 Pre-match model

Features: `[elo_diff, form_diff, h2h, toss_bat_first, toss_field_first]` — 5 features. Two calibrated classifiers (isotonic-calibrated Logistic Regression and Gradient Boosting) are compared against a climatology baseline (always predicting the training-set base rate).

Pre-match model — Brier Skill Score vs. climatology

BSS = 0 $\rightarrow$ no better than the trivial base-rate baseline. Negative = worse than it.



Both trained models have negative BSS — they are measurably worse than just predicting the league base rate. This single number is the strongest piece of evidence in the whole codebase for the standard-ML papers' blind spot: papers reporting 80–94% "accuracy" on pre-match prediction without benchmarking against a trivial baseline or checking calibration.

8.2 Dynamic in-game models

Two separate models, one per innings: **2nd innings** (DYN2 : `runs_needed, balls_remaining, wkts_remaining, crr, rrr, elo_adv, phase`) predicts `chasing_wins` ; **1st innings** (DYN1 : `team_runs, team_wicket, balls_remaining, run_rate, proj_total, elo_adv, phase`) predicts `defending_wins` .

Dynamic in-game AUC vs. pre-match AUC

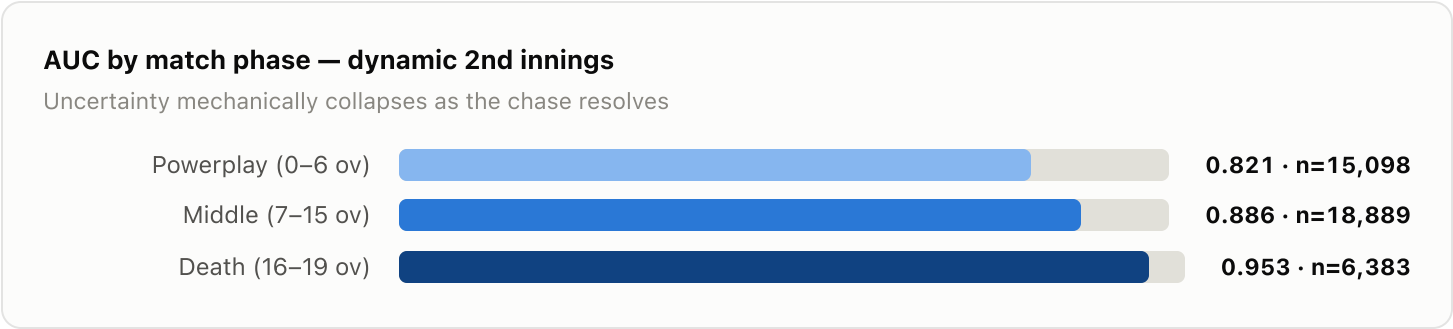
1.0 = perfect ranking · 0.5 = no better than random



Dynamic 2nd-innings AUC (0.878) is **+0.375** over pre-match's 0.5035 — the single largest effect size in the whole results set: in-game state carries almost all of the predictive signal in T20 cricket.

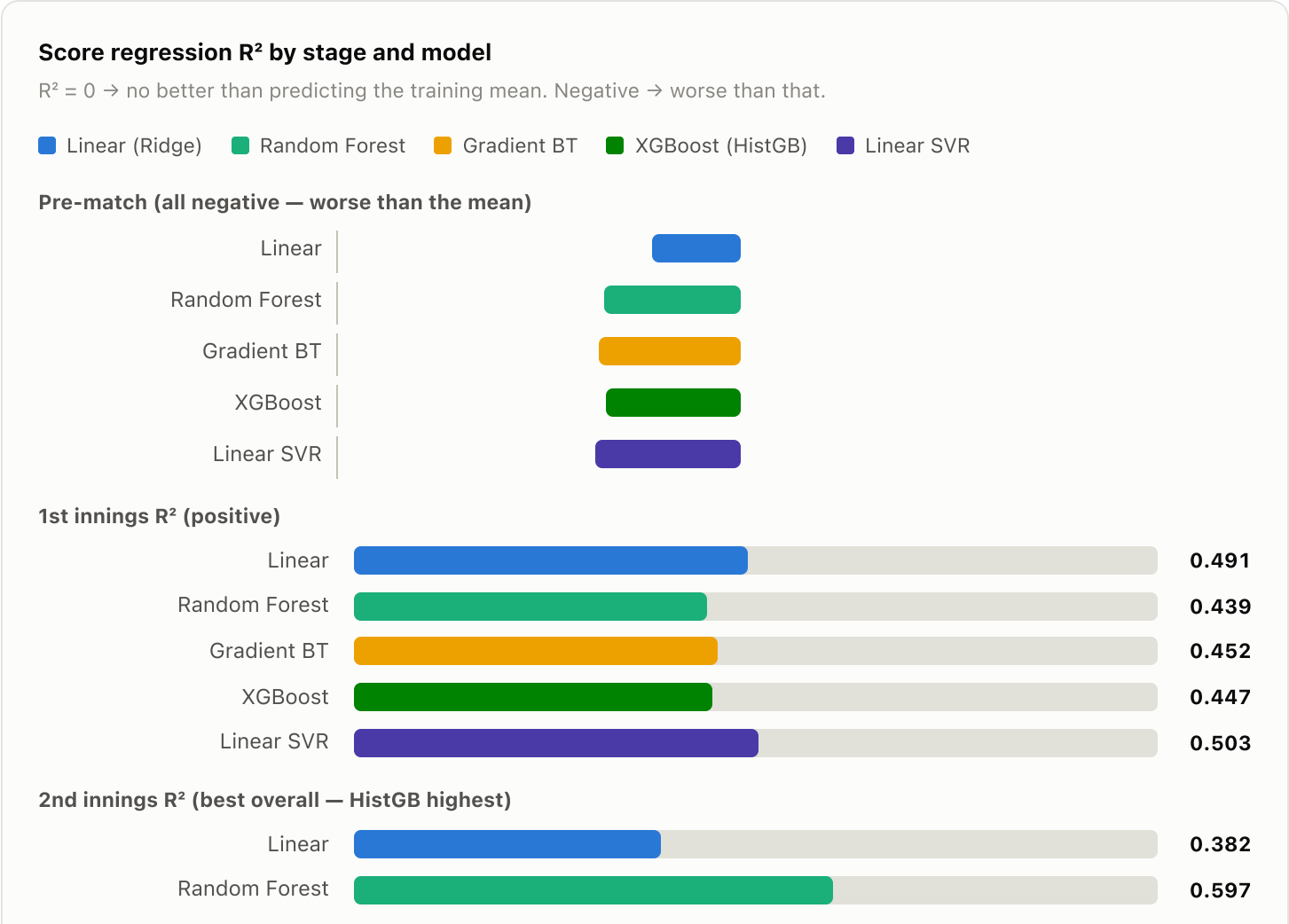
8.3 Phase-specific evaluation

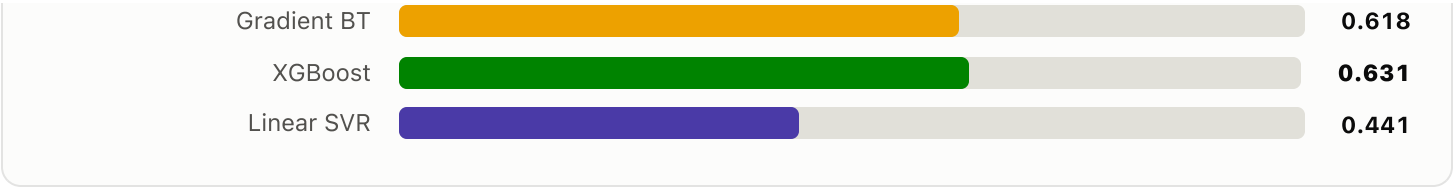
Refits a separate calibrated model per phase (Powerplay / Middle / Death) using the same feature set minus phase itself.



8.4 Score regression zoo

Five regressors (Ridge, Random Forest, Gradient Boosting, HistGradientBoosting-as-"XGBoost", Linear SVR) fit separately at three stages: pre-match, 1st innings, 2nd innings.

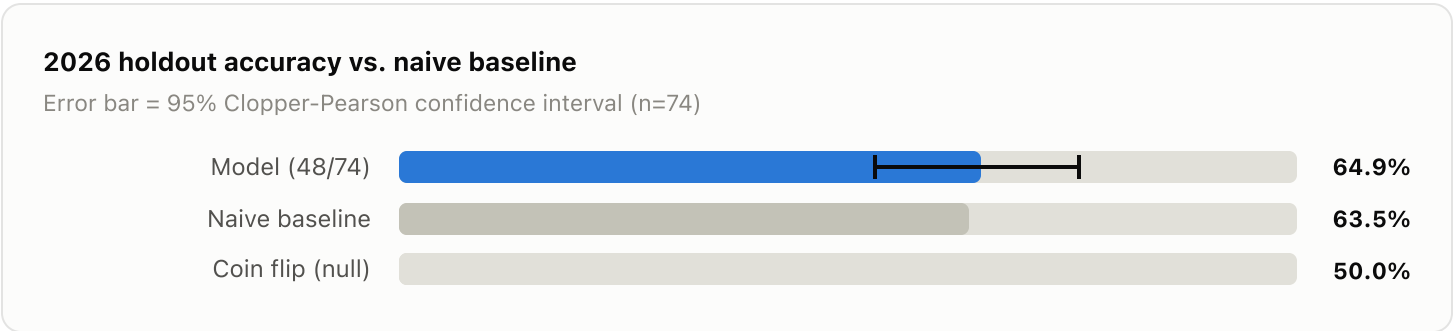




No single model wins across all three stages — Linear SVR is best pre-match and 1st innings, HistGB best 2nd innings.

8.5 The locked 2026 holdout

Not just "predict on held-out rows" — a genuine walk-forward simulation: retrain on the full 2008–2025 dataset, walk through the 74 locked 2026 matches in chronological order updating Elo/form/H2H after each result, and report accuracy with a Clopper-Pearson 95% CI plus a z-test p-value against the 50/50 null.



METRIC	VALUE
Accuracy	64.9% (48/74)
95% CI (Clopper-Pearson)	[52.9%, 75.6%]
p-value vs. 50/50 null	0.011
Naive baseline	63.5%
Margin over naive baseline	+1.4 points

Statistically significant vs. a coin flip, but the 95% CI is wide enough (a 22.7-point span) that it cannot distinguish "genuine skill" from "the naive baseline plus noise" at this sample size.

8.6 Calibration-method comparison

Isotonic regression (the pipeline's default) vs. temperature scaling (`src/temperature_scaling.py`), on a genuinely held-out validation slice (2019–2020) to fit temperature's scalar `T`, so neither method sees its own evaluation data during fitting.

9 Temperature scaling — `src/temperature_scaling.py`

A from-scratch implementation of Guo et al. (2017)'s single-parameter calibration method: find scalar $T > 0$ minimising validation negative-log-likelihood of $\text{sigmoid}(\text{logit}(p) / T)$. $T > 1$ softens overconfident predictions; $T < 1$ sharpens underconfident ones.

Isotonic vs. temperature scaling — dynamic 2nd innings

Temperature $T = 1.0906$ (raw model was slightly overconfident)

Brier score (lower is better)

Temp-scaled: raw		0.1453
Temp-scaled: cal.		0.1449
Isotonic (default)		0.1446

ECE (lower is better)

Temp-scaled: raw		0.0458
Temp-scaled: cal.		0.0448
Isotonic (default)		0.0436

Both methods move ECE in the right direction; isotonic ends slightly ahead on all three metrics, consistent with it being the pipeline's chosen default rather than an arbitrary choice.

10 Secondary model: causal Transformer + MC-Dropout

A clearly-labeled **secondary, non-default** win-probability path — its own prior evaluation found it does not beat the calibrated Logistic Regression baseline at a statistically significant level.

Architecture (`IPLTransformer`): a causal (autoregressive-masked) Transformer encoder over the 120-dimensional input (24-dim game state $\oplus$ 3 $\times$ 32-dim player embeddings for batter/bowler/non-striker), 2 layers, 4 heads, `d_model=64` .

Multi-task design, deliberately disabled by default: the loss function supports three heads — win probability (BCE), next-ball outcome (7-way cross-entropy), and projected final score (Huber loss) — but ships with `lambda_next_ball=0`, `lambda_score=0` (win-only), because prior evaluation found the auxiliary heads actively hurt the win-probability signal.

Player embeddings: a fixed, randomly-initialised (Xavier) `nn.Embedding` , not a learned or graph-pretrained representation — a prior ablation found no significant benefit from graph-based pretraining at this data scale.



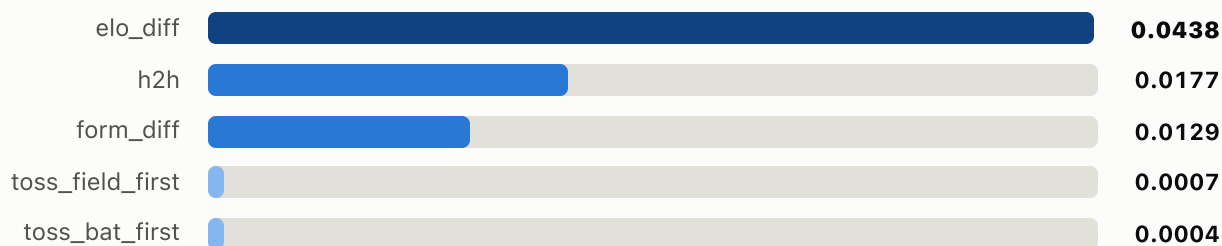
MC-Dropout uncertainty (`WinProbabilityEngine`): at inference, dropout is deliberately kept *active* and the model is run `MC_SAMPLES=50` times per prediction; the variance across those stochastic forward passes is used as an epistemic-uncertainty estimate (`mean  $\pm$  1.96  $\cdot$  std`) — the single closest thing in the codebase to a genuine Bayesian treatment.

11 Explainability — `src/explainability.py`

SHAP permutation-explainer values for the **calibrated** classifier's `predict_proba` output — not the raw uncalibrated base estimator, since `CalibratedClassifierCV`'s isotonic layer would otherwise be invisible to a naive SHAP run. Applied to the pre-match Cal. GBT model.

SHAP feature importance — pre-match Cal. GBT model

Mean |SHAP value| across explained test predictions



Elo alone carries roughly two-thirds of the total SHAP mass across the five features; toss carries under 2% combined — a direct, quantified answer to whether toss matters for pre-match prediction (it barely does, once Elo is present).

12 Tournament simulation — `src/tournament.py`

Builds and compares **actual vs. model-predicted** 2026 points tables (Net Run Rate included, computed the standard cricket way). Purely a downstream consumer of the pre-match model's per-match predictions — no new modeling, but a genuinely useful sanity check at the tournament level rather than just per-match.

CHECK	RESULT
Actual points-table topper	Royal Challengers Bengaluru
Predicted points-table topper	Royal Challengers Bengaluru — correct
Top-4 overlap (actual n predicted)	3 of 4 teams (GT, PBKS, RCB)

A more intuitively meaningful aggregate check than a bare 64.9% match-level accuracy figure, since a model that's individually noisy on 74 matches can still be systematically right about which teams are strongest overall.

13 Evaluation-metric definitions — `src/metrics.py`

Three functions, deliberately built so the scalar and the diagnostic plot data can never disagree:

- `calibration_bins(y_true, y_prob, n_bins=10)` — per-bin reliability-diagram data (predicted-mean vs. observed-frequency per bin, empty bins omitted).
- `ece(...)` — Expected Calibration Error, built strictly on top of `calibration_bins` (no independent recomputation).
- `brier_skill_score(y, p, p_clim)` — requires an explicit training-set climatology array rather than computing it from the test set in-place, specifically to avoid leaking test-set label distribution into the reference forecast.



Recently fixed bug, worth noting for methodological transparency: `np.linspace(0,1,n_bins+1)` makes the final bin's upper edge exactly 1.0; the original half-open interval convention meant predictions of exactly `p=1.0` (which isotonic calibration produces routinely at saturation) matched no bin at all and were silently dropped. Fixed by closing the last bin on both ends. Verified impact: 1,020 of 40,370 dynamic-2nd-innings test predictions were affected, moving the reported dyn2 ECE from 0.0430 to 0.0434 — small here, but the class of bug generalises to any pipeline using isotonic calibration.

14 Presentation layer — dashboard

A single self-contained HTML file (no external JS dependency, no CDN — every chart is hand-rolled SVG) that renders every result above as an interactive reliability diagram, trajectory chart, Elo-over-time line chart, phase/AUC bar chart, and tournament table, plus a **live in-browser win-probability calculator** that re-implements the trained dynamic 2nd-innings model's exact math in JavaScript, verified to match the real Python model's `predict_proba` to full float precision. `dashboard_export.py` merges freshly computed pipeline results into the page's embedded `DATA` JSON blob without touching hand-authored content elsewhere in the file, so every number on the dashboard is traceable to an actual `run_all.py` run.

15 Test suite — tests/

Tests organised so that: (a) every scalar formula (`phase` , `crr` , `rrr` , Elo update, Beta-shrinkage math) has direct unit tests with hand-computed expected values; (b) integrity tests check monotonicity/boundedness invariants on synthetic data mirroring the real schema; (c) `test_game_state.py` / `scripts/validate_game_state.py` cross-check the 24-dim feature vector against the real dataset's own derived columns on 5 real matches chosen to exercise no-ball/wide edge cases; (d) "model sanity" tests train tiny synthetic-data models and assert they've learned obvious cricket logic (fewer wickets remaining → lower win probability) — a check that catches "the code runs but learned nothing sensible" bugs that pure shape/type tests would miss.

16 Results — the full numbers, consolidated

Every table below is pulled directly from a real `run_all.py` execution against the actual dataset (1,146 matches, 273,503 deliveries, train ≤ 2020 / test > 2020 internal split unless noted).

16.1 Pre-match win prediction (internal, n=347 test matches)

MODEL	BRIER ↓	AUC	ACCURACY	BSS	ECE ↓
Climatology (baseline)	0.2508	0.5000	0.4813	0.0000	0.0000
Calibrated Logistic Regression	0.2637	0.4527	0.4755	-0.0516	0.1092
Calibrated Gradient Boosting	0.2559	0.5035	0.4986	-0.0204	0.0654

16.2 Dynamic (in-game) win prediction

MODEL	TEST SET	BRIER ↓	AUC	ACCURACY	ECE ↓
Dynamic 2nd innings (chasing wins)	40,370 deliveries	0.1445	0.8782	0.7918	0.0434
Dynamic 1st innings (defending wins)	—	0.2203	0.7022	0.6395	—

16.3 Phase-specific dynamic 2nd-innings models

PHASE	OVERS	AUC	BRIER ↓	N (TEST DELIVERIES)
Powerplay	0–6	0.8210	0.1784	15,098
Middle	7–15	0.8857	0.1381	18,889
Death	16–19	0.9534	0.0836	6,383

16.4 Score regression — MAE (runs) / R^2 by stage and model

MODEL	PRE-MATCH	INN. 1	INN. 2
Linear (Ridge)	30.9 / -0.219	18.7 / 0.491	19.6 / 0.382
Random Forest	32.6 / -0.338	19.1 / 0.439	14.0 / 0.597
Gradient Boosting	32.8 / -0.350	18.9 / 0.452	13.6 / 0.618
HistGB ("XGBoost")	32.8 / -0.335	18.9 / 0.447	13.3 / 0.631

Linear SVR	32.8 / −0.361	18.4 / 0.504	17.8 / 0.440
------------	---------------	--------------	--------------

16.5 Locked 2026 external holdout (74 matches, walk-forward)

METRIC	VALUE
Accuracy	64.9% (48/74)
95% CI (Clopper-Pearson)	[52.9%, 75.6%]
p-value vs. 50/50 null	0.011
Naive baseline (always pick historically-favoured side)	63.5%
Margin over naive baseline	+1.4 points

16.6 SHAP feature importance (pre-match Cal. GBT model)

FEATURE	MEAN SHAP
elo_diff	0.0438
h2h	0.0177
form_diff	0.0129
toss_field_first	0.0007
toss_bat_first	0.0004

16.7 Tournament-level simulation (2026 season)

CHECK	RESULT
Actual points-table topper	Royal Challengers Bengaluru
Predicted points-table topper	Royal Challengers Bengaluru — correct
Top-4 overlap	3 of 4 (GT, PBKS, RCB)

16.8 Calibration method comparison — dynamic 2nd innings

METHOD	BRIER (RAW → CAL.)	AUC (RAW → CAL.)	ECE (RAW → CAL.)
Temperature scaling (T=1.0906)	0.1453 → 0.1449	0.8777 → 0.8777	0.0458 → 0.0448
Isotonic (pipeline default)	— → 0.1446	— → 0.8783	— → 0.0436

16.9 An unconsumed result worth noting: `make_zoo()`

`src/models.py` defines a 5-classifier win-probability zoo (Logistic, Random Forest, Gradient Boosting, HistGB, LinearSVC), but `grep` confirms it is **never called anywhere in the codebase**. Only the regression equivalent (`make_score_zoo()`, §16.4) is actually used — the classification tables above use two hand-built classifiers constructed directly in `pipeline.py`. Wiring `make_zoo()` into the classification tasks would directly answer "does a stronger classifier close the pre-match gap, or is the ceiling feature-driven?" with real numbers instead of speculation.

17 Honest limitations

1. **Pre-match win/score prediction is close to useless** as currently featured — both classification (AUC ~0.51 LOSO) and regression (negative R^2) confirm this independently.
2. **No hierarchical/Bayesian team-strength model.** Elo is a point estimate with no shrinkage for new/sparse teams; the Beta-smoothed H2H is the only genuinely Bayesian element in the codebase.
3. **Player-level features exist but are unconsumed** by any trained model — the highest-leverage, lowest-effort extension available.
4. **The secondary Transformer's numbers are not directly comparable** across any two runs unless trained on identical data splits.
5. **74-match 2026 holdout is small.** The Clopper-Pearson CI is wide (52.9–75.6%) specifically because of this.
6. **No opponent-adjusted or venue-adjusted scoring model** — no Poisson/bivariate-Poisson analogue of the football literature.
7. **The 5-classifier win-probability zoo (`make_zoo()`) is dead code.** Only 2 of its 5 classifiers are ever actually trained via hand-built calibrated estimators in `pipeline.py` .

18 Suggested framing for a research paper

A. Calibration-first evaluation of standard ML on IPL win prediction

Use §8.1/8.5/§13's results as the empirical core. Demonstrate concretely that IPL pre-match models reported with only accuracy fail a climatology/BSS baseline, and that a naive "usually-favoured team wins" baseline nearly matches a trained model's 2026 holdout accuracy.

B. From point-estimate Elo to Bayesian team strength

Use §4/§8.1's Elo implementation as the baseline to beat. Implement the missing shrinkage layer and measure whether it improves predictions specifically for new/sparse franchises (GT, LSG).

C. MC-Dropout vs. true Bayesian inference for in-game win probability

Use §10's MC-Dropout engine as one arm, implement a genuinely Bayesian dynamic logistic regression (PyMC/NumPyro) on the same `DYN2` features as the other arm, and quantify how much MC-Dropout's approximate uncertainty diverges from exact posterior uncertainty on identical data.

Any of the three is scoped to use existing, tested, working code as the control/baseline arm — meaning the paper's novel contribution is isolated to one clearly-bounded extension rather than requiring a rebuild from scratch.